

**ME 2261 - Embedded Systems Log Book**Task 01

Write AVR assembly program to create approx. 1 millisecond delay using conditional jump instructions without using built-in delay functions or timers.

Assuming:

CPU Clock: 8MHz

Each instruction takes 1 cycle (unless branching)

We'll use nested loops and conditional branches.

Frequency = 1/Time taken for 1 clock cycle

$$\text{Time taken for 1 clock cycle} = \frac{1}{8 \times 10^6} \text{s} = 0.125 \mu\text{s}$$

Maximum no. of loops that can be perform using 8-bit register = 255

$$\text{Time taken for 1 loop} = \frac{1000 \mu\text{s}}{255} = 3.92 \mu\text{s}$$

$$\text{No. of cycles required for 1 loop} = \frac{3.92 \mu\text{s}}{0.125 \mu\text{s}} \sim 31 \text{ cycles}$$

No. of instructions required for 1 loop = 31

$$\text{Total No. of clock cycles} = 31 \times 255 = 7905$$

$$\text{Total Time taken} = 7905 \times 0.125 \mu\text{s} = 988.125 \mu\text{s} \sim 1 \text{ s}$$

$$\text{No. of Constant instructions required} = \frac{1000 - 988.125}{0.125} = 95 \text{ (I've used only 2)}$$

But the Branching instruction clock cycle can be 2 mostly.  
so the time delay of 1 ms can be achieved approximately.

```

1  eor r27,r27
2  loop:inc r27
3      cpi r27,255
4      nop
5      nop
6      nop
7      nop
8      nop
9      nop
10     nop
11     nop
12     nop
13     nop
14     nop
15     nop
16     nop
17     nop
18     nop
19     nop
20     nop
21     nop
22     nop
23     nop
24     nop
25     nop
26     nop
27     nop
28     nop
29     nop
30     nop
31     brlo loop
32     nop

```

| Processor       |                 |
|-----------------|-----------------|
| Name            | Value           |
| Program Counter | 0x0000001F      |
| Stack Pointer   | 0x0000          |
| X Register      | 0xFF00          |
| Y Register      | 0x0000          |
| Z Register      | 0x0000          |
| Status Register | I T H S V N Z C |
| Cycle Counter   | 7905            |
| Frequency       | 8.000 MHz       |
| Stop Watch      | 988.13 µs       |
| Registers       |                 |
| R00             | 0x00            |
| R01             | 0x00            |
| R02             | 0x00            |
| R03             | 0x00            |
| R04             | 0x00            |
| R05             | 0x00            |
| R06             | 0x00            |
| R07             | 0x00            |
| R08             | 0x00            |
| R09             | 0x00            |
| R10             | 0x00            |
| R11             | 0x00            |

### Task 1

To make output signal of 1 and 0 for each of 5 clock cycles.

```
1  LDI R16,0x20
2  LDI R17,0x00
3  OUT DDRD,R16    // Set DDR5 as Output
4  loop:
5  OUT PORTD,R16   // Set Output 1
6  NOP
7  NOP
8  NOP
9  NOP
10 OUT PORTD,R17   // Set Output 0
11 NOP
12 JMP loop       // Takes 3 clock cycles
```

### Task 2

To make output signal of 1 and 0 for each of 1 ms time interval.

For CPU Clock: 8MHz

Each instruction takes 1 cycle (unless branching)

Calculation for the number of instructions are made similar to the first task, where loop

```
1  .INCLUDE "M32ADEF.INC" // Atmega32 microcontroller
2  .org $00
3
4  begin:
5  LDI R16,0x20
6  LDI R17,0x00
7  OUT DDRD,R16    // Set DDR5 as Output
8  loop:
9      EOR R18,R18
10     OUT PORTD,R16 // Set Output 1
11     loop1:inc R18
12         nop nop nop nop nop nop nop nop nop
13         nop nop nop nop nop nop nop nop nop
14         nop nop nop nop nop nop nop nop nop
15         cpi R18,250
16         brlo loop1
17     EOR R18,R18
18     OUT PORTD,R17 // Set Output 0
19     loop2:inc R18
20         nop nop nop nop nop nop nop nop nop
21         nop nop nop nop nop nop nop nop nop
22         nop nop nop nop nop nop nop nop nop
23         cpi R18,250
24         brlo loop2
25     JMP loop     // Takes 3 clock cycles
26 END:NOP
```

The .hex file of the AVR Program is flashed into the Atmega32A microcontroller using eXtreme Burner – AVR. Then PD5 Pin and GND pin is connected with channel 1 of Digital Oscilloscope, and got the graph like below: Indicating the amplitude voltage and period of a pulse of 5V and 1000  $\mu$ s.

